



BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri

Veritabanı Güvenliği ve Erişim Kontrolü

GitHub: <https://github.com/ismailbsrn/DatabaseScenarios-2>

Video: https://drive.google.com/file/d/168e6W5nPMTUnAK_ULezhM975Zj-tUTQd/view?usp=share_link

İsmail Başaran
22290137

Nisan 2026

1. Projenin Amacı

Bu projenin temel amacı, örnek bir veri tabanı kullanarak veri tabanı güvenliğine dair konular hakkında testler ve iyileştirmeler yapmaktır. Bu projede örnek veri tabanı olarak internette kolaylıkla bulunabilecek olan DVD Rental veri tabanı kullanılacaktır. Veri tabanı yönetim sistemi olarak ise PostgreSQL kullanılacaktır. Çalışma macOS işletim sistemi üzerinde gerçekleştirilecektir.

2. Proje Kapsamı

Proje aşağıdaki özellikleri içermektedir:

- PostgreSQL'in kendi erişim kontrolcüsü ile kullanıcı erişim yetkileri yönetimi.
- PostgreSQL veri tabanlarında yaygın olarak kullanılan pgcrypto aracı ile sütun bazlı şifreleme.
- Veri tabanı seviyesinde SQL Injection testlerinin yapılması ve saldırı önleme.
- Veri tabanı üzerinde yapılan değişikliklerin takibi amacıyla audit loglarının kullanımı.

3. Gerçekleştirim ve Yöntem

Bahsedilen aşamaların nasıl gerçekleştirildiği detaylarıyla birlikte bu bölümde açıklanmaktadır.

3.1. Erişim Yönetimi

Erişim yönetimi veri tabanları için hayati öneme sahiptir. Kullanıcılara ihtiyaçları olabilecek en düşük yetkileri vermek (Principle of Least Privilege - PoLP) bilgi güvenliğinin gereklilikleri arasındadır. Raporun bu aşamasında bu senaryo gerçekleştirilecektir.

Erişim yönetimi senaryosunun gerçekleştirilmesi için aşağıdaki PostgreSQL sorgusu ile iki farklı rol oluşturulmuştur.

```
dvdrental=# create role read_only_role;
CREATE ROLE
dvdrental=# create role read_write_role;
CREATE ROLE
```

Oluşturulan her iki role de veri tabanına bağlanma ve şemalara erişim yetkisi verilmiştir.

```
dvdrental=# grant connect on database dvdrental to read_only_role, read_write_role;
GRANT
dvdrental=# grant usage on schema public to read_only_role, read_write_role;
GRANT
```

Sonrasında read_only rolüne yalnızca okuma yetkisi verilmiştir.

```
dvdrental=# grant select on all tables in schema public to read_only_role;
GRANT
```

read_write rolüne ise CRUD yetkileri verilmiştir.

```
dvdrental=# grant select, insert, update, delete on all tables in schema public to read_write_role;
GRANT
dvdrental=#
```

Senaryo gereği bu rollerin atanacağı Rapor Uzmanı ve Yönetici kullanıcıları oluşturulmuş ve ilgili roller atanmıştır.

```
dvdrental=# create user rapor_uzmani with password 'guclusifre123';
CREATE ROLE
dvdrental=# create user yonetici with password 'yoneticisifresi123';
CREATE ROLE
dvdrental=# grant read_only_role to rapor_uzmani;
GRANT ROLE
dvdrental=# grant read_write_role to yonetici;
GRANT ROLE
```

Bu hazırlık işlemlerinden sonra veri tabanına Rapor Uzmanı kullanıcısı ve şifresi ile bağlanılmıştır.

```
DatabaseScenarios-2 $ psql -U rapor_uzmani -d dvdrental
Password for user rapor_uzmani:
psql (18.3 (Homebrew))
Type "help" for help.

dvdrental=> select first_name, last_name from customer limit 5;
 first_name | last_name
-----+-----
Jared      | Ely
Mary       | Smith
Patricia   | Johnson
Linda      | Williams
Barbara    | Jones
(5 rows)

dvdrental=> delete from customer where customer_id=1;
ERROR:  permission denied for table customer
dvdrental=>
```

Görüleceği üzere Rapor Uzmanı veri tabanından veri okuyabilirken silme işlemi yapamamaktadır.

Sonrasında veri tabanına Yönetici kullanıcısı ve şifresi ile bağlanılmıştır.

```
DatabaseScenarios-2 $ psql -U yonetici -d dvdrental
Password for user yonetici:
psql (18.3 (Homebrew))
Type "help" for help.

dvdrental=> select first_name, last_name from customer limit 5;
 first_name | last_name 
-----+-----
Jared      | Ely
Mary       | Smith
Patricia   | Johnson
Linda      | Williams
Barbara    | Jones
(5 rows)

dvdrental=> update customer set first_name='YENI_ISIM' where customer_id=1;
UPDATE 1
```

Yönetici kullanıcı CRUD yetkilerine sahip olduğunda tablo üzerinde hem okuma hem de yazma işlemi yapabilmektedir. Böylelikle PostgreSQL veri tabanlarında Erişim Yetkisi Yönetimi gerçekleştirilmiştir.

3.2. Şifreleme

Veri tabanındaki verilerin şifreli olarak saklanması veri güvenliği açısından oldukça önemlidir. Verilerin şifreli şekilde saklanması olası bir veri sızıntısı durumunda kötü niyetli kişilerin tüm verilere ulaşmasını imkânsız kılar. Ancak bu durumda anahtarın gizliliğine dikkat edilmelidir.

Raporun bu kısmında PostgreSQL veri tabanında veri şifreleme süreçlerinin nasıl gerçekleştirildiği incelenmiştir. PostgreSQL veri tabanı dahili varsayılan olarak SQL Server'daki disk şifreleme özelliğine sahip değildir. Disk şifreleme için Linux tarafında LUKS (Linux Unified Key Setup) kullanılmaktadır. Bu bağlamda projemizde sütun bazlı şifrelemeye imkân tanıyan pgcrypto eklentisi kullanılmıştır.

İlk olarak en yetkili kullanıcı olan superadmin rolündeki postgres kullanıcısı ile örnek veri tabanımız olan DVDrental veri tabanına bağlanılmış ve ardından pgcrypto eklentisi kurulmuştur.

```
DatabaseScenarios-2 $ psql -U postgres -d dvdrental
psql (18.3 (Homebrew))
Type "help" for help.
```

```
dvdrental=# create extension if not exists pgcrypto;
CREATE EXTENSION
```

Sonrasında customer tablosuna “encrypted_email” adında yeni bir sütun eklenmiş ve bu sütunun veri tipi olarak Byte Array seçilmiştir.

Ardından customer tablosu güncellenmiş ve belirlenen şifre ile şifrelenen veriler encrypted_email sütununa yazılmıştır.

```
dvdrental=# alter table customer add column encrypted_email bytea;
ALTER TABLE
dvdrental=# update customer set encrypted_email = pgp_sym_encrypt(email, 'guclusifre123');
UPDATE 599
dvdrental=#
```

Tablo güncellendikten sonra şifrelenmemiş e-posta adreslerinin olduğu email sütunu silinmiştir. Bu sayede gizlenmek istenen verinin şifrelenmemiş hali veri tabanından silinmiştir.

```
dvdrental=# alter table customer drop column email;
ALTER TABLE
```

Aşağıdaki görselde görüleceği üzere veriler herhangi bir şifre çözme işlemi uygulanmadan okunmaya çalışırsa e-posta adreslerine erişilememektedir. Ancak doğru şifre sağlandığında tüm veriler görüntülenebilmektedir.

```
DatabaseScenarios-2 $ psql -U postgres -d dvdrental
psql (18.3 (Homebrew))
Type "help" for help.

dvdrental=# select first_name, last_name, encrypted_email from customer limit 5;
 first_name | last_name | encrypted_email
-----+-----+-----
Jared       | Ely       | \xc30d0407030281539f5d66748f967cd24d01296593cc72649af85c4e6f3f2693d7973facf3ca8074b3cd6d5f03abe9b5c3d01cf446eb897f9308f93
c0e5c3e9cf11b9233037e87c32b37ee2c385e299e774851e1d4d4c54b1aab0807dee7c
Patricia    | Johnson   | \xc30d040703022c59e02c7fa8cf9674d254014bb51667fea39c65fe186f53517289d5c505a42982931d6ac1a8dec2bcd8942c7f1bc2a5930f6455ed
e4d7ad7397e1cc029bde0b550893808be0a3a1c45419538efe4227955f386991b76890150df778aa7b4
Linda       | Williams  | \xc30d040703026ed09bbbc4ede0e86dd2520117bd6d6873e3120b40e08d0b63f08ac5cdd2ddabc8b032ac35e9d46962ee1746e453e8f17ba4db96cb1
56d30db1243fd05e6f1b5d3111b83700a32eca0919d218fb6a712d1b3c49e71d549253d79f6f73
Barbara     | Jones     | \xc30d04070302c7f57665a41edc6b60d251010d1e55b9e937d5401aac514c3c41b029dbff18ba69db87606bda50811c32e3558273dc4004a70694bb2
e94c011947c4480f0ac18d93aacdabe90893d11dccc51c862dcfbc3e7f035e50422f6290a8868
Elizabeth   | Brown     | \xc30d04070302b29711d3a7cd300a7ad253018dc55e4de7c50974f88a25be20ea497bc46a895fda263a7bdf57eb9441d1febfa4a0fc36315b7039c94
d002380038791cd3d3ecc052877c8c84ea33e33be0a0c5b74bafce6f3d5cd7ca0668c197e107560b4a
(5 rows)

dvdrental=# select first_name, last_name, pgp_sym_decrypt(encrypted_email, 'guclusifre123') from customer limit 5;
 first_name | last_name | pgp_sym_decrypt
-----+-----+-----
Jared       | Ely       | jared.ely@sakilacustomer.org
Patricia    | Johnson   | patricia.johnson@sakilacustomer.org
Linda       | Williams  | linda.williams@sakilacustomer.org
Barbara     | Jones     | barbara.jones@sakilacustomer.org
Elizabeth   | Brown     | elizabeth.brown@sakilacustomer.org
(5 rows)

dvdrental=#
```

Yanlış şifre ile erişim denendiğinde ise şifrenin yanlış olduğu uyarısıyla karşılaşılmaktadır.

```
dvdrental=# select first_name, last_name, pgp_sym_decrypt(encrypted_email, 'yanlissifre') from customer limit 5;
ERROR: Wrong key or corrupt data
```

3.3. SQL Injection

SQL Injection veri tabanı sistemlerine karşı gerçekleştirilen saldırılarda en çok kullanılan yöntemlerden biridir. Kötü niyetli kişiler veri tabanı sorgusu içine dahil edilecek metinler içerisine ekledikleri tırnaklar için sorguda kendilerine izin verilen alanın dışına çıkarlar ve OR '1' == '1 benzeri ifadelerle ulaşmamaları gereken verilere ulaşırlar. Aşağıda bu duruma örnek bir durum incelenmiştir.

Bir web uygulamasında veri tabanından veri çekmek için aşağıdaki sorgunun kullanıldığını varsayalım.

```
query = f"SELECT first_name FROM customer WHERE email = '{email}';"
```

Bu durumda kullanıcının sağladığı e-posta adresi veri tabanından kullanıcının adını sorgulamak için kullanılmaktadır. Ancak bu sorgu kötü niyetli kişiler tarafından manipüle edilmeye müsaittir.

```
string' OR '1'='1
```

Kötü niyetli kişiler e-posta alanına herhangi bir e-posta adresi girmek yerine bu yapıda veri girerse çalıştırılan sorgu

```
query = f"SELECT first_name FROM customer WHERE email = 'string' OR '1'='1';"
```

halini alır ve e-postanın ne olduğundan bağımsız olarak veri tabanındaki tüm kullanıcı adlarına erişilmiş olur.

Raporun bu kısmında Python ve FastAPI kullanılarak bir SQL Injection senaryosu hazırlanmış ve gerçekleştirilmiştir.

Öncelikle aşağıdaki kod aracılığıyla SQL Injection saldırılarına karşı korumasız bir endpoint oluşturulmuştur. Kullanıcıdan alınan metnin direkt olarak sorgu içerisine ekleniyor olması sistemi savunmasız kılmaktadır.

```
@app.get("/api/vulnerable/customer")
def get_customer_vulnerable(first_name: str):
    conn = get_db_connection()
    cur = conn.cursor()
    query = f"SELECT first_name, last_name FROM customer WHERE first_name = '{first_name}';"
    try:
        cur.execute(query)
        results = cur.fetchall()
        return {"calisan_sorgu": query, "sonuclar": results}
    except Exception as e:
        return {"hata": str(e)}
    finally:
```

```
conn.close()
```

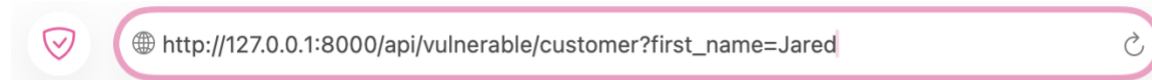
Ardından aşağıdaki kod ile SQL Injection saldırılarına karşı güvenli bir endpoint oluşturulmuştur. Parametrik sorgular ile metin direkt sorgu içerisine enjekte edilmemektedir. psycopg2 kütüphanesi metin içerisindeki tırnakları escape eder ve sonrasında sorgu içerisine ekler. Bu sayede tırnak sembolü kullanılarak SQL Injection yapmak imkânsız hale gelir.

```
@app.get("/api/secure/customer")
def get_customer_secure(first_name: str):
    conn = get_db_connection()
    cur = conn.cursor()
    query = "SELECT first_name, last_name FROM customer WHERE first_name = %s;"
    try:
        cur.execute(query, (first_name,))
        results = cur.fetchall()
        return {"calisan_sorgu": query, "sonuclar": results}
    except Exception as e:
        return {"hata": str(e)}
    finally:
        conn.close()
```

Backend hazırlandıktan sonra aşağıdaki komut ile backend ayağa kaldırılmıştır.

```
(venv) DatabaseScenarios-2 $ uvicorn main:app --reload --env-file .env
```

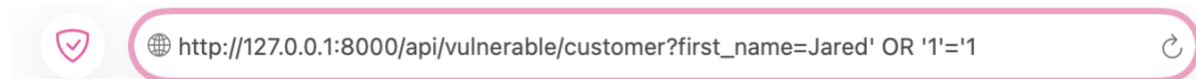
Sonrasında tarayıcıda aşağıdaki adres girilmiştir.

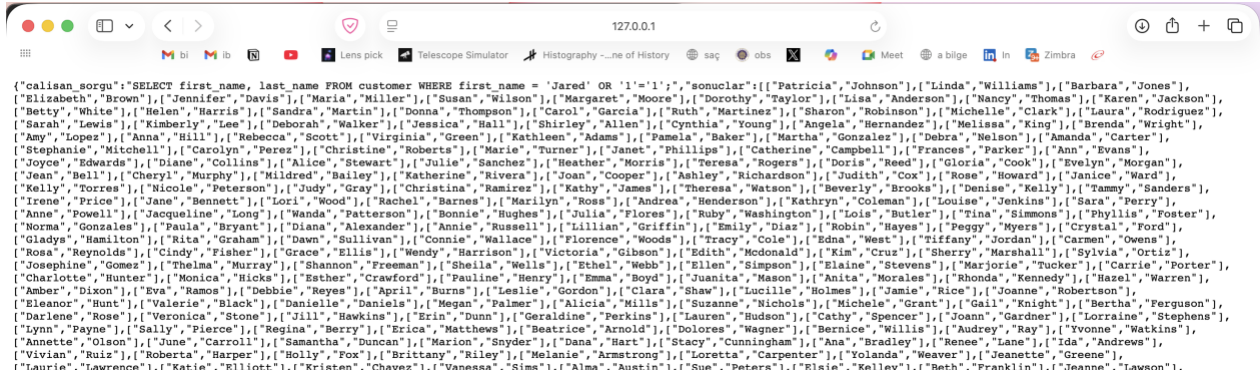


Bu işlem sonucunda aşağıdaki yanıt elde edilmiştir.

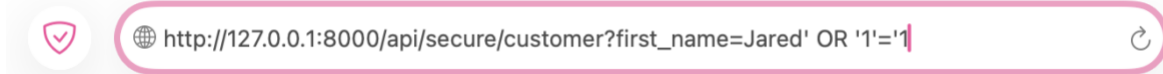
```
{"calisan_sorgu": "SELECT first_name, last_name FROM customer WHERE first_name = 'Jared';",
"sonuclar": [["Jared", "Ely"]]}
```

Bu işlem oldukça masum durmaktadır ancak aşağıdaki sorgu çalıştırıldığında neden öyle olmadığı görülecektir.





Görüldüğü gibi SQL Injection kullanılarak tüm isimlere erişilmiştir.



```
{"calisan_sorgu":"SELECT first_name, last_name FROM customer WHERE first_name = %s;", "sonuclar":[]}
```

Aynı sorgu güvenli olan endpointte denendiğinde ise ismi tam olarak “Jared ‘ OR ‘1’=’1” olan kullanıcılar aranmaktadır ve böyle bir kullanıcı olmadığından boş bir liste dönmektedir.

4. Audit Logları

Veri tabanı üzerinde yapılan değişikliklerin takibi amacıyla audit logları kullanılmaktadır. Raporumuzun bu kısmında PostgreSQL veri tabanlarında sunucu seviyesinde audit takibinin nasıl yapılandırıldığını inceleyeceğiz.

Öncelikle audit logları için pgaudit eklentisini kuruyoruz.

```
(venv) DatabaseScenarios-2 $ psql -U postgres -d dvdrental
psql (18.3 (Homebrew))
Type "help" for help.
```

```
dvdrental=# create extension if not exists pgaudit;
CREATE EXTENSION
```

Sonrasında pgauditin hangi işlemleri loglayacağını belirtiyoruz.

```
dvdrental=# alter system set pgaudit.log='write';
ALTER SYSTEM
```


Sistem tablolarının loglanmasını istemediğimiz için log_catalog değerini off yapıyoruz.

```
dvdrental=# alter system set pgaudit.log_catalog = off;  
ALTER SYSTEM
```

Parametre değerlerinin de loglanmasını istediğimiz için log_parameter değerini on yapıyoruz.

```
dvdrental=# alter system set pgaudit.log_parameter = on;  
ALTER SYSTEM
```

Ayarların oturuma yansması için config dosyamız üzerinden reload işlemi gerçekleştiriyoruz.

```
dvdrental=# select pg_reload_conf();  
pg_reload_conf  
-----  
t  
(1 row)
```

Ardından takip etmek istediğimiz roller için hangi işlemlerin takip edileceğini belirtiyoruz.

```
dvdrental=# alter role satis_elemani set pgaudit.log = 'write';  
ALTER ROLE
```

Bu işlem sonrasında şu anki zamanı sorgulayıp not alıyoruz ve ardından bir UPDATE işlemi gerçekleştiriyoruz.

```
(venv) DatabaseScenarios-2 $ psql -U satis_elemani -d dvdrental  
psql (18.3 (Homebrew))  
Type "help" for help.  
  
dvdrental=> select current_time;  
current_time  
-----  
14:00:43.940863+03  
(1 row)  
  
dvdrental=> update customer set activebool = false where customer_id = 15;  
UPDATE 1  
dvdrental=> █
```

Log dosyalarını incelediğimizde belirtilen zamanda bir UPDATE işlemi gerçekleştirildiğini ve bu işlemin hangi kullanıcı tarafından hangi veri tabanı üzerinde gerçekleştirildiğini görüyoruz.

```
2026-04-14 14:00:37.227 +03 [63047] @:LOG: database system is ready to accept connections  
2026-04-14 14:00:47.839 +03 [63060] satis_elemani@dvdrental:LOG: AUDIT: SESSION,1,1,WRITE  
,UPDATE,,,update customer set activebool = false where customer_id = 15,<none>
```

5. Çıktılar

Proje kapsamında yapılan işlemler sayesinde PostgreSQL veri tabanlarının güvenliğini artırmak için neler yapılabileceğini, veri tabanının nasıl şifrelenebileceğini, SQL Injectiona karşı ne gibi önlemler alınabileceğini, Audit Logları ile değişiklik takibinin nasıl yapılacağını öğrendim.